Problem A: A History of Computing

1. (1936): The Turing Machine – Alan Turing provided the theoretical foundation of today's computer systems by proving that one computer can perform all algorithms.

2. (1947): The Transistor – By utilizing semiconductors instead of big and delicate vacuum tubes, this invention paved the way for hardware miniaturization and was the building block of all today's electronics.

3. (1958): The Integrated Circuit (Microchip) – Enabled many transistors to be etched onto a single slice of silicon, drastically boosting computing capabilities while reducing prices (directly leading to Moore's law).

4. (1969): ARPANET & Apollo 11 – ARPANET was used to test packet switching (which became the basis of today's internet), whereas the Apollo 11 Guidance Computer demonstrated that software could perform in real time.

5. (1981): The IBM PC – It was the creation of an open hardware standard that moved computers out of their huge corporate mainframe incarnation to become a household computer.

6. (1991): The World Wide Web & Linux – Tim Berners-Lee makes the WWW publicly available, and Linus Torvalds releases the first Linux kernel, which was an open sourced and cooperative OS which was able to compete with the big commercial OS in the market.

7. (2000): The Y2K Bug – The panic surrounding the year 2000 exposed the fragility of global digital infrastructure, forcing a massive worldwide software audit and standardizing modern, robust coding practices thus revolutionising the entire coding and tech development.

8. (2007): The iPhone – Apple releases the first iPhone, fundamentally changing mobile computing, app ecosystems, and user interfaces.

9. (2012): AlexNet (Deep Learning) – A neural network called AlexNet crushes the ImageNet competition, sparking the modern AI and deep learning boom.

10. (2022): ChatGPT (Generative AI) – OpenAI releases ChatGPT, bringing advanced Large Language Models into mainstream public use.

Problem B: Pixel Art

The problem here wants us to build patterns on an *N* x *N* grid as stated in the question itself where N is and input. We are to build two primary art pieces, a checker box and a diamond [rhombus] as given by the user as input. The constrains are as follows : -

1. The grid size (N × N grid, 5 ≤ N ≤ 51, always odd for the diamond shape)
2. shape: Either "checkerboard" or "diamond".
3. The top-left cell is always 0.

We need to print [or return in original skeleton of code] that *N* x *N* grid with the white pixels being marked as 0, and the dark pixels being marked as 1, and the accumulation on the grid must represent the said shape.

NOTE: the answer is explain from the perspective of a function and the answer is returned as a variable with the name 'grid' which is originally filled with 0s.

➔ std::vector<std::vector<int>> grid(n, std::vector<int>(n, 0));

THE FULL CODE IMPLEMENTATION TO GIVEN SKELETON IS ATTACHED AS WELL IN A 'CPP' FILE WITH THIS SUBMISSION

HANDLING THE CHECKERBOARD

First we look into the checker-board problem. We know from constrains and regulations that the top left corner MUST be a '0'. A **naïve** approach would be to iterate through the grid using two nested loops and use an explicit if/else condition to check if the sum of the coordinates (i + j) is even or odd, assigning 1 or 0 accordingly.

My solution: However the time complexity can be improved by just removing the conditional 'if' statement and assigning the mathematical result of (i + j) % 2 to the cell DIRECTLY. The modulus of 2 gives binary results and distinct from even or odd! Separating it into a different if statement would result to not better value in this scenario.

```
if(shape == "checkerboard"){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            grid[i][j] = (i+j)%2;
        }
    }
```

```
        }
```

The above code snippet gives us a better idea as to how it works

HANDLING THE DIAMOND

Naïve: The diamond is usually done by breaking up the shape into 4 different shapes and broken into several pieces to solve manually for every part of the shape [mostly broken in triangles]. This results in a terrible time complexity due to the manual breaking and looping of the positions.

My Solution: The better way to approach this is to follow the mathematical formula for each cell where we check ' abs(n/2-i) + abs(n/2-j) <= n/2 '

This gives us the exact coordinates where the '1's should be placed

```
else if(shape == "diamond"){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            if(abs(n/2-i) + abs(n/2-j) <= n/2){
                grid[i][j] = 1;
            }
        }
    }
}
```

The above code snippet should make things clearer.

The entire code is given below [and is the exact replica of the c++ code file that is attach along with this submission].

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
```

```
/**
 * Generates a geometric pattern on an n x n grid.
 *
 * Args:
 *     n:     Grid size (n x n, always odd for diamond)
 *     shape: Either "checkerboard" or "diamond"
 *
 * Returns:
 *     A 2D vector of integers (0 or 1).
 */
std::vector<std::vector<int>> generate_shape(int n, const std::string& shape) {

   // WRITE YOUR CODE HERE
   std::vector<std::vector<int>> grid(n, std::vector<int>(n, 0));
   if(shape == "checkerboard"){
      for(int i = 0; i < n; i++){
         for(int j = 0; j < n; j++){
            grid[i][j] = (i+j)%2;
         }
      }
   }
   else if(shape == "diamond"){
      for(int i = 0; i < n; i++){
         for(int j = 0; j < n; j++){
            if(abs(n/2-i) + abs(n/2-j) <= n/2){
               grid[i][j] = 1;
            }
         }
```

```
        }
    }
    return grid;
}

// --- Main execution block. DO NOT MODIFY ---
int main() {
    try {
        std::string line;
        std::getline(std::cin, line);
        int n = std::stoi(line);

        std::string shape;
        std::getline(std::cin, shape);

        auto result = generate_shape(n, shape);
        for (int i = 0; i < (int)result.size(); i++) {
            for (int j = 0; j < (int)result[i].size(); j++) {
                if (j > 0) std::cout << " ";
                std::cout << result[i][j];
            }
            std::cout << "\n";
        }

    } catch (const std::exception& e) {
        std::cerr << "An unexpected error occurred: " << e.what() << std::endl;
        return 1;
    }
```

```
    return 0;
}
```

Problem C: Moore's Law

Moore's Law is the law stated my Gordon Moore in 1975, about the exponential nature of the increase in the use of transistors.

Formula: $\mathbf{T(t) = T_0 \times 2^{(t/2)}}$

where T0 is the transistor count at a reference year, and t is the number of years elapsed since that reference.

(a) Though the Moore's law was stated in 1975, stating from the perspective of 1971, we try to shoot a shot at predicting the "transistor COUNT for a microchip of the year 2026". So let us apply the formula

→ **$\underline{T(55) = 2300 + 2^{(55/2)}}$** | ∵ t -> is 55 years passed (1971 – 2026 = 55), and the first processor [the one that we are to consider] has 2300 transistors

➔ **∴** Predicted Transistors in 2026: **<u>436568821872 [ANS]</u>**

The solution was calculated by the following code : -

```
#include <iostream>
#include <cmath>
#include <iomanip>

int main() {
   double T0 = 2300.0;
   double t = 2026.0 - 1971.0; // 55 years elapsed

   // Moore's Law Formula: T(t) = T0 * 2^(t/2)
   double predicted_transistors = T0 * std::pow(2.0, t / 2.0);

   std::cout << std::fixed << std::setprecision(0);
   std::cout << "Predicted Transistors in 2026: " << predicted_transistors << "\n";

   return 0;
}
```

∵ **solution for (a) = 436,568,821,872 [that is 436.5 BILLION transistor prediction]**

(b) Now let's see the actual number of transistors in a modern 2026 microchip processor. We shall take two examples. Firstly,

a. Apple M5 -> has 28 Billion transistors
b. the record holder as of March 2025 is **Apple's ARM-based dual-die M3 Ultra SoC**, featuring **184 billion transistors**

As of 2026, the Record holder for the greatest number of transistors in a microprocessor still is Astronomically far behind from the original prediction!

Even if we consider a super sophisticated GPU with a record breaking **336 billion MOSFETs,** Nvidia's **Rubin accelerator** is still far behind our calculations!

∴ Moore's Law doesn't hold up at ALL after these 55 years of development! At least not in the field of microprocessors.

<u>Suspected reason</u>: Although Moore's law was accurate in its initial predictions of exponential growth in computing, this growth rate has been **decelerating significantly** because of numerous **physical and financial limitations**. The transistors are becoming so small at the current 3nm to 5nm scale that there is a **problem of quantum tunnelling** in physics. Additionally, creating processors with **400+ billion transistor**s would make their **cost of production prohibitively high** for the mass market. Lastly, despite the **need of the software like AI** for large processing power**, the size of the processor can no longer increase due to heat emission and efficiency of daily use**.

CONCLUSION: Moore's Law does NOT hold after 55 years of technological advancements due to the above stated susceptions.

(c) We know the start (2300), the end ($28{,}000{,}000{,}000$), and the time (55 years). We need to find p (the doubling period) in the equation:

$$28{,}000{,}000{,}000 = 2300 \times 2^{55/p}$$

The code we use to find the soln to the above equation is as follows : -

```
#include <iostream>

#include <cmath>

#include <iomanip>


int main() {
```

```
    // Known variables
    double T_actual = 28000000000.0; // 28 billion transistors (Apple M5)
    double T0 = 2300.0;              // Transistors in 1971
    double t = 55.0;                 // Years elapsed (2026 - 1971)

    // Mathematical derivation:
    // T_actual = T0 * 2^(t/p)
    // T_actual / T0 = 2^(t/p)
    // log2(T_actual / T0) = t / p
    // p = t / log2(T_actual / T0)

    double ratio = T_actual / T0;
    double p = t / std::log2(ratio);

    // Print result rounded to 2 decimal places
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "The actual doubling period (p) is: " << p << " years\n";

    return 0;
}
```

The answer comes to: **2.34 years**

Problem D: The Simultaneous Strike

Lets look at the case first before we try to answer. We see that we are having a dispute for a double KO. The game rules state that the match is calculated by EVERY SINGLE FRAME. This means that every frame MUST BE RESOLVED before moving on[**this is a key clue**].

Lets take a look at the pseudocode

Input: events: list of (player, frame, attack_value) in arbitrary order (due to network jitter)
Output: player1_hp, player2_hp

1: stable sort events by frame number

2: for all event in events do

3: if player1_hp ≤ 0 OR player2_hp ≤ 0 then

4: break

5: end if

6: opponent_hp ← opponent_hp - event.attack_value

we note that there was network jitter

(a) One of the main problems that game devs HAVE to tackle is network packet loss. The ping of players is the deciding matter in many such cases. Let me explain. **PING** is the measurement of latency, specifically the time it takes for data to travel from your device to the game server and back, expressed in **milliseconds (ms)**. This is one of the **deciding** factors for game mechanics calculations especially when it comes to multiplayer damage dealing. We see a hint of the game having jittery network. We suspect issues on the pseudo code.

*issue in a broad scope: **UNLESS both players HAVE SENT the EXACT same packet on the EXACT same ping on the EXACT same frame, the server will NOT be in sync, the reason is as follows!**

Deduction: We see a MAJOR flaw in the server code. It checks the event machinations **SEQUENTIALLY,** which means that it doesn't resolve the Frame first but rather its checking in a sequential **stable_sorted** manner, which translates to : **The server is checking the State of the player of 'n'th frame WHO REACHES FIRST, and NOT simultaneously!!!**

This means that player1 ‘s game signal of a hit HAS REACHED the server BEFORE player2 ‘s signal AND SINCE the server had the major flaw of checking sequentially instead of simultaneously, the server **AWARDED** player1 BECAUSE he reached the server before player2.

Thus we deduce that the issue of the server is that it is checking the game Event logs in a sequential manner instead of a simultaneous manner, which goes AGAINST the code of conduct for the game, which explicitly states “ALL EVENTS MUST BE RESOLVED ON FRAME BEFORE PROCEEDING”.

The Conclusion for (a): -

Player 1's signal reached the server milliseconds before Player 2's signal. Because the server checks **sequentially**, it instantly awarded the win to Player 1 and triggered the break statement. Player 2's packet—which was tagged with the exact same frame—**was completely ignored**. This explicitly violates the game's rule that "all events on the same frame must be applied before checking for a KO." *The server **MUST** check all the **accumulation of events** per frame, before moving on or breaking from the loop in this case.*

(b) The problem with sequential damage has been fixed by sorting game events by frame tag in the modified processGame function, the changes are as follows.

- Chronological Sorting: Network events have been sorted by frame using std::stable_sort, allowing us to put a chaotic order of packet arrivals into a chronological sequence while maintaining original order in case of equality.

- Frame-by-Frame Processing (Outer Loop): The outer loop moves along the timeline of the game. Most importantly, it performs a KO check (hp1 <= 0 || hp2 <= 0) when entering a new frame to allow the game to end right away if anyone had a death in the previous frame.

- Simultaneous Damage (Inner Loop): The inner loop collects all events whose currentCheckFrame is equal. All the damage from this particular frame is processed simultaneously before the KO check of the outer loop.

- Health Points Clamping: The HPs are clamped by std::max(0, hp), which solves the Double KO scenario effortlessly.

the below code snippet is an EXACT replica of the code given in the code file

```
#include <algorithm>
#include <iostream>
#include <map>
#include <vector>
using namespace std;

/**
 * @param events Vector of {player, frame, attack_value}
 * @param H      Starting HP for both players
 * @return vector {hp1, hp2} each clamped to min 0
 */

vector<int> processGame(vector<vector<int>> events, int H) {

  // WRITE YOUR CODE HERE
  // Sort events chronologically by frame number (which is index 1 of the inner
  // vector)
  std::stable_sort(
      events.begin(), events.end(),
      [](const vector<int> &a, const vector<int> &b) { return a[1] < b[1]; });
  int hp1 = H, hp2 = H;
  int f = 0;
  while (f < events.size()) {

    // Check if anyone died from the PREVIOUS frame
    if (hp1 <= 0 || hp2 <= 0) {
      break;
    }
```

```
    // Process all attacks for the CURRENT frame
    int currentCheckFrame = events[f][1];
    while (f < events.size() && events[f][1] == currentCheckFrame) {
      events[f][0] == 1 ? hp2 -= events[f][2] : hp1 -= events[f][2];
      f++;
    }
  }

  // THE CLAMP: Forces any negative numbers to become 0
  return {std::max(0, hp1), std::max(0, hp2)};
}

// --- Main execution block. DO NOT MODIFY ---
int main() {
  try {
    int H, n;
    cin >> H >> n;
    vector<vector<int>> events(n, vector<int>(3));
    for (int i = 0; i < n; i++) {
      cin >> events[i][0] >> events[i][1] >> events[i][2];
    }

    vector<int> result = processGame(events, H);
    cout << result[0] << " " << result[1] << endl;

  } catch (const exception &e) {
    cerr << "Error: " << e.what() << endl;
    return 1;
  }
```

```
  return 0;
}
```

CONLUCSION FOR QUESTION D

The solution thus provided has the concept of “Frame wise resolution” where **each** frame is resolved before moving to another event. This keeps the games fair and always in sync and removes the errors that network jitters bring forth! This is a full proof solution to the game in the question.

Problem E: PageRank in the Age of AI Slop

This problem brings forth a historical problem and a current standpoint together to a convergence. We are to realise the depth of pageranks and Google's key to raise to the top

(a) **My initial thoughts (a wrong path):** Understanding the formula's recursive nature was simple, as we see that for each page's rank development, we need to find the PageRank of the page pointing to it (p_i). This continues until we reach a certain endpoint. ***This led me to view this situation as a classic recursive tree formation. While the formulation is recursive, the NATURE OF THE INTERNET IS NOT A CLEAN TREE***—there are loops and chaos all over. I originally thought 'd' was a damping 'force' to prevent chaos from messing up the rankings due to the changing and chaos of the internet. However, using standard dynamic programming (memoization) doesn't solve the infinite void of looped references (spider traps), distorting the whole concept of the 'random-surfer interpretation'.

THE ACTUAL ANSWER: -

- The Random-Surfer: A page's PageRank is **simply the mathematical probability** that a random surfer *mindlessly clicking links* lands on that specific page at any given moment.
- The Recursive Nature: ***A page's rank is determined by the rank of the pages linking to it***. You cannot mathematically know Page A's rank without knowing Page B's rank, requiring **a recursive definition**.
- The Damping Factor (d): This is **NOT** for smoothing out a tree structure. It represents the **exact chance** (usually 85%) **that the surfer clicks a link**. The remaining 15% $(1-\mathrm{d})$ is the chance they "teleport" to a random page. This teleportation is what mathematically rescues the surfer from infinite loops.
- How it is Solved (Power Iteration): Google's answer was not memoization. They assign every page a starting guess (usually $1/\mathrm{N}$), run the formula for every single page simultaneously to generate new scores, and repeat that massive loop until the numbers stop changing (convergence).

(b) To find the total PageRank of all AI pages (S_A), we just add up the three "buckets" where they get their rank.

1. The Base Bucket (Random Teleportation)

Every single page on the internet gets a baseline score of $\frac{1-d}{N}$.

Since there are a AI pages, their total base score is $a \times \frac{1-d}{N}$.

Because $\alpha = \frac{a}{N}$, this simplifies to:

Base = $\alpha(1-d)$

2. The Human Bucket (Inflow from Humans)

Human pages have a total rank of S_H. They link completely randomly across the entire internet. Since AI pages make up a fraction (α) of the internet, they receive exactly that fraction of the human rank.

Since total rank is 1, $S_H = (1 - S_A)$.

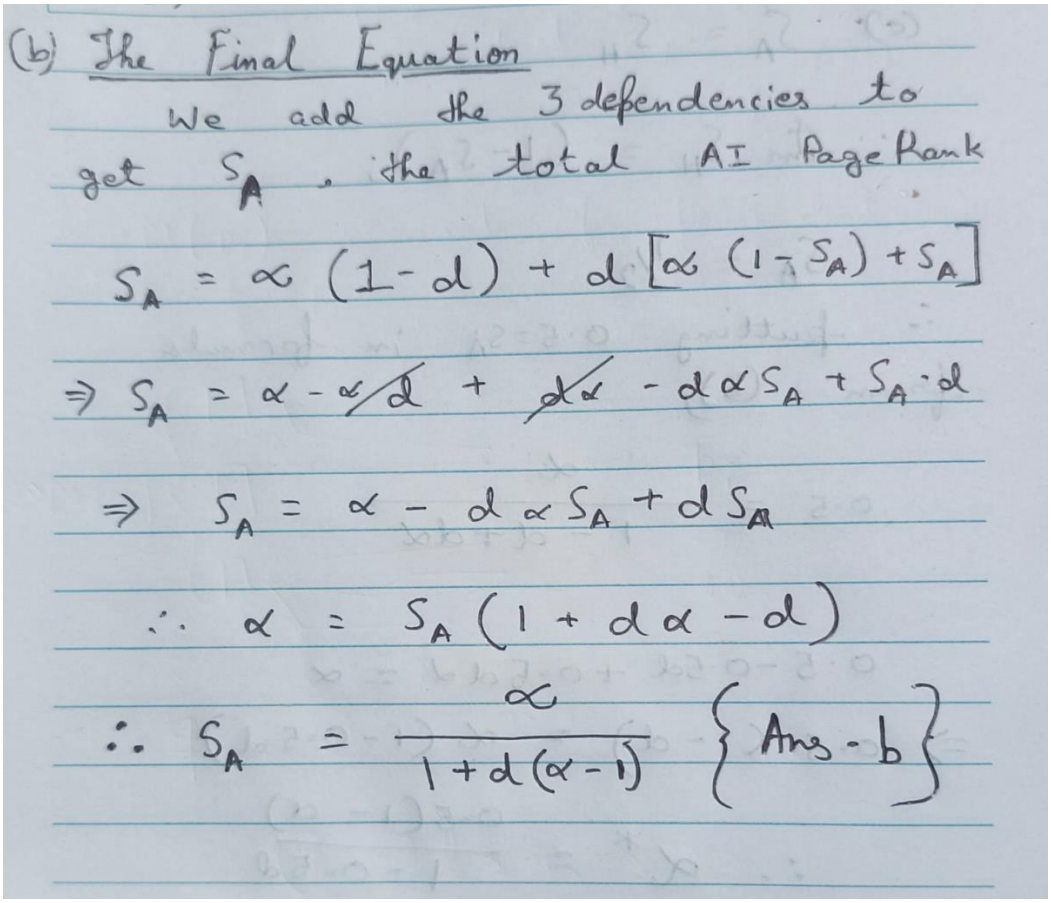
Human Inflow = $d \times \alpha(1 - S_A)$

3. The AI Bucket (Inflow from AI)

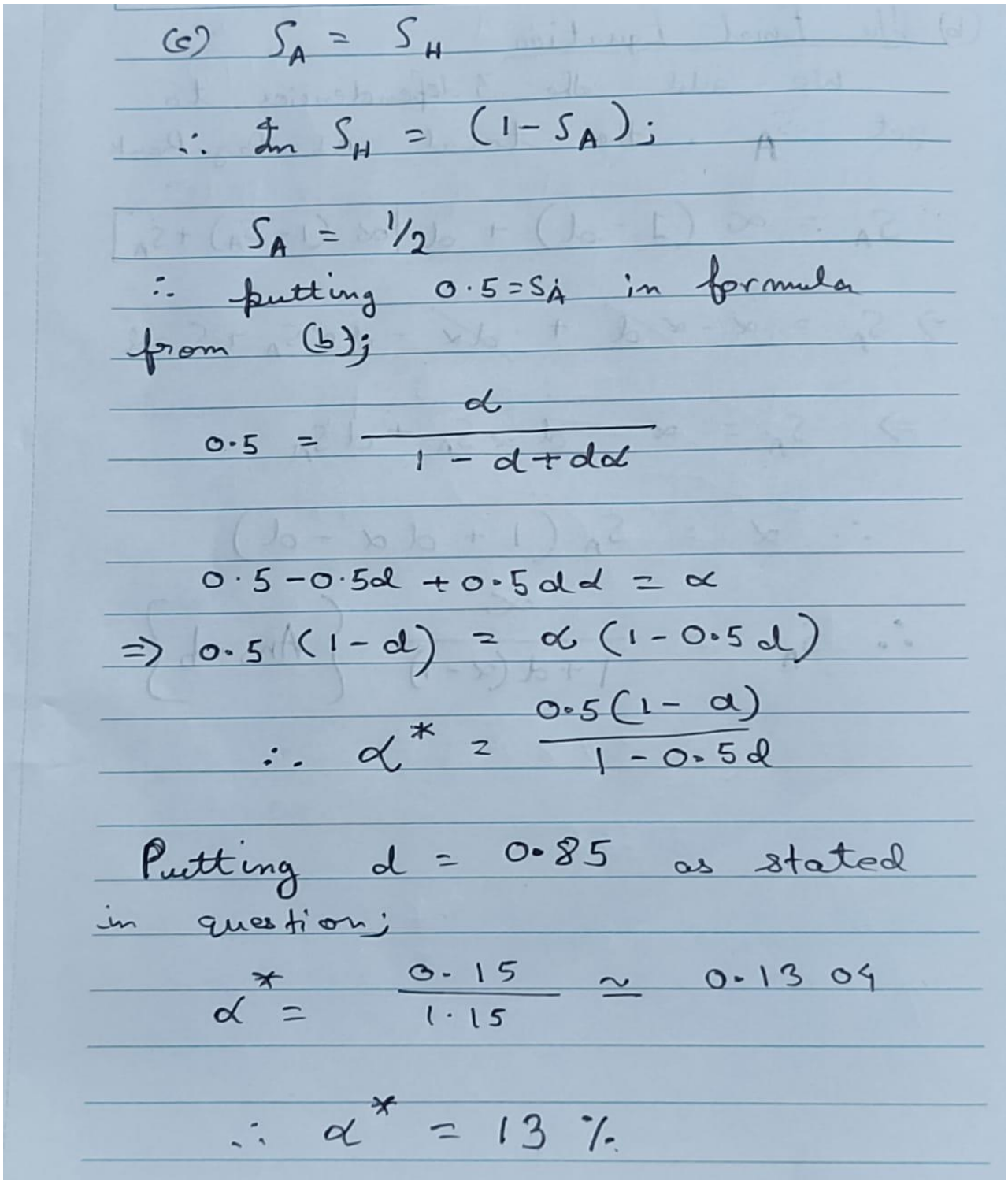
AI pages *only* link to other AI pages. They trap 100% of their own rank in an endless loop.

AI Inflow = $d \times S_A$

<u>The final solving of the equation is written by hand and is as follows</u>



(c) The solution starts off with the equation itself, and noting that $S_A = S_H$, Is given in the question. And we have to find the tipping fraction



Answer: The tipping fraction is roughly 13%.

Conclusion

This math proves that PageRank is incredibly vulnerable to "link farms" (or AI slop pages) that trap links. Because AI pages selfishly hoard their outgoing links and human pages link randomly, the AI network only needs to make up **13%** of the

internet to steal **50%** of the total search authority. Google's original algorithm naturally funnels power to pages that refuse to share it.